

Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Mestrado Integrado em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

Fase 3: Curvas, Superfícies Cúbicas e VBOs

Jorge Rafael Machado Fernandes
A104168

Diogo Teixeira Fernandes
A104260

Filipe Teixeira Viana
A104361

CG

Data da Receção	
Responsável	
Avaliação	
Observações	

Fase 3: Curvas, Superfícies Cúbicas e VBOs

maio, 2026

Índice

1. Introdução	1
2. Generator	2
2.1. Superfícies de Bézier	2
2.1.1. Polinómios de Bernstein	2
2.1.2. Avaliação de um Patch	2
2.1.3. Tesselação	2
2.2. Parsing do Ficheiro .patch	3
3. Engine	4
3.1. Evolução da Estrutura da Cena	4
3.1.1. Tipos de Transformação	4
3.1.2. ModelData e VBOs	4
3.2. Transformações Dinâmicas	4
3.2.1. Translação por Curvas de Catmull-Rom	5
3.2.2. Alinhamento à Curva	6
3.2.3. Rotação Baseada em Tempo	6
3.3. Renderização com VBOs	7
3.3.1. Criação dos Buffers	7
3.3.2. Desenho dos Modelos	7
3.4. Exploração da Cena	7
4. Cena de Demonstração: Sistema Solar Dinâmico	8
5. Utilitários e Estruturas de Dados	10
6. Conclusões e Trabalho Futuro	11

Lista de Figuras

Figura 1	Vista aproximada do sistema solar com o cometa (teapot de Bézier) visível	8
Figura 2	Vista geral do sistema solar dinâmico com a trajetória de Catmull-Rom do cometa	9

1. Introdução

Este relatório foi elaborado no âmbito da Unidade Curricular de Computação Gráfica, no ano letivo de 2025/2026, e documenta o desenvolvimento da terceira fase do trabalho prático.

Nesta fase, estendemos o generator e o engine desenvolvidos nas fases anteriores, introduzindo três novas funcionalidades fundamentais: a geração de modelos a partir de superfícies cúbicas de Bézier, o suporte a transformações dinâmicas baseadas em tempo (curvas de Catmull-Rom e rotações temporais), e a migração da renderização para Vertex Buffer Objects (VBOs).

A funcionalidade implementada foi validada através da construção de uma cena dinâmica do sistema solar, que demonstra a aplicação de todas as novas capacidades, incluindo um cometa modelado a partir de patches de Bézier cuja trajetória é definida por uma curva de Catmull-Rom.

2. Generator

O generator desenvolvido nas fases anteriores permitia gerar primitivas geométricas simples (esfera, cone, plano, caixa). Nesta fase, foi adicionado suporte para a geração de modelos a partir de ficheiros de patches de Bézier.

2.1. Superfícies de Bézier

Uma superfície de Bézier bicúbica é definida por 16 pontos de controlo organizados numa grelha **4×4**. Cada ponto da superfície é calculado através da avaliação paramétrica nos parâmetros u e v , ambos no intervalo $[0, 1]$.

2.1.1. Polinómios de Bernstein

A base de Bernstein de grau 3 é composta por quatro polinómios:

$$B_0(t) = (1 - t)^3$$

$$B_1(t) = 3t(1 - t)^2$$

$$B_2(t) = 3t^2(1 - t)$$

$$B_3(t) = t^3$$

Estes polinómios formam uma partição da unidade, ou seja:

$$B_0(t) + B_1(t) + B_2(t) + B_3(t) = 1, \quad \forall t \in [0, 1]$$

Esta propriedade garante que os pontos avaliados se encontram dentro do invólucro convexo dos pontos de controlo.

2.1.2. Avaliação de um Patch

Dado um patch com 16 pontos de controlo P_{ij} ($i, j = 0, \dots, 3$), um ponto na superfície para os parâmetros (u, v) é calculado por:

$$S(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) \cdot B_j(v) \cdot P_{ij}$$

Cada coordenada (x, y, z) é avaliada independentemente, aplicando o mesmo somatório ponderado a cada componente dos pontos de controlo.

2.1.3. Tesselação

Para converter a superfície contínua numa malha de triângulos, o patch é subdividido numa grelha de **tessellation × tessellation** quadriláteros. Para cada quadrilátero definido pelos parâmetros (u_1, v_1) , (u_2, v_1) , (u_1, v_2) e (u_2, v_2) , onde:

$$u_1 = \frac{i}{\text{tessellation}}, \quad u_2 = \frac{i+1}{\text{tessellation}}$$

$$v_1 = \frac{j}{\text{tessellation}}, \quad v_2 = \frac{j+1}{\text{tessellation}}$$

São avaliados quatro pontos na superfície:

$$P_1 = S(u_1, v_1), \quad P_2 = S(u_2, v_1)$$

$$P_3 = S(u_1, v_2), \quad P_4 = S(u_2, v_2)$$

Cada quadrilátero é dividido em dois triângulos:

- **Triângulo 1:** P_1, P_3, P_2
- **Triângulo 2:** P_2, P_3, P_4

O número total de triângulos gerados é:

$$N_{\text{triângulos}} = 2 \times n_{\text{patches}} \times \text{tessellation}^2$$

Para o teapot com 32 patches e tesselação 10, são gerados $2 \times 32 \times 100 = 6400$ triângulos, correspondendo a 19200 vértices.

2.2. Parsing do Ficheiro .patch

O formato do ficheiro .patch é composto por:

- Número de patches (primeira linha)
- Para cada patch: uma linha com 16 índices separados por vírgulas, referenciando os pontos de controlo
- Número de pontos de controlo
- Para cada ponto: coordenadas x, y, z separadas por vírgulas

No ficheiro XML, o comando do generator é invocado como:

```
generator bezier <ficheiro.patch> <tessellation> <output.3d>
```

Exemplo:

```
generator bezier teapot.patch 10 teapot.3d
```

3. Engine

3.1. Evolução da Estrutura da Cena

A principal alteração ao nível da estrutura de dados foi a reformulação do sistema de transformações e do armazenamento dos modelos.

3.1.1. Tipos de Transformação

Na fase anterior, as transformações eram pré-calculadas numa única matriz acumulada 4×4 durante o parsing. Nesta fase, as transformações passaram a ser armazenadas individualmente e aplicadas em tempo real durante a renderização, permitindo transformações dependentes do tempo.

Foram definidos cinco tipos de transformação:

Tipo	Descrição	Parâmetros
TRANSLATE_STATIC	Translação fixa	x,y,z
TRANSLATE_CATMULLROM	Translação por curva de Catmull-Rom	time, align, pontos de controlo
ROTATE_STATIC	Rotação fixa	angle, x, y, z
ROTATE_TIME	Rotação dependente do tempo	time (segundos para 360°), x, y, z
SCALE	Escala fixa	x, y, z

Tabela 1: Tipos de transformação suportados pelo engine

Cada grupo mantém uma lista ordenada de transformações que são aplicadas sequencialmente durante o desenho.

3.1.2. ModelData e VBOs

A estrutura ModelData foi reformulada para suportar VBOs:

- **vbold (GLuint)**: identificador do buffer OpenGL que contém a geometria na GPU
- **vertexCount (int)**: número de vértices do modelo
- **r, g, b (float)**: cor do modelo, normalizada para [0, 1]

A geometria deixou de ser armazenada em RAM como vetores de pontos, passando a residir inteiramente na memória da GPU após o carregamento.

3.2. Transformações Dinâmicas

O tempo decorrido desde o início da aplicação é obtido através de `glutGet(GLUT_ELAPSED_TIME)`, convertido para segundos:

$$\text{elapsed} = \frac{\text{glutGet}(\text{GLUT_ELAPSED_TIME})}{1000.0}$$

Este valor é utilizado para calcular as transformações dependentes do tempo.

3.2.1. Translação por Curvas de Catmull-Rom

Uma curva cúbica de Catmull-Rom permite interpolar suavemente entre um conjunto de pontos de controlo. Dados quatro pontos consecutivos P_0, P_1, P_2, P_3 , o ponto na curva para o parâmetro $t \in [0, 1]$ é dado por:

- $Q(t) = T \cdot M \cdot P$

onde:

- $$T = \begin{pmatrix} t^3 \\ t^2 \\ t \\ 1 \end{pmatrix}^T$$
- $$M = \begin{pmatrix} -0.5 & 1.5 & -1.5 & 0.5 \\ 1.0 & -2.5 & 2.0 & -0.5 \\ -0.5 & 0.0 & 0.5 & 0.0 \\ 0.0 & 1.0 & 0.0 & 0.0 \end{pmatrix}$$
- $$P = \begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \end{pmatrix}$$

A derivada da curva, utilizada para o alinhamento, é calculada substituindo T pelo vetor:

- $$T' = \begin{pmatrix} 3t^2 \\ 2t \\ 1 \\ 0 \end{pmatrix}^T$$

Para mapear um parâmetro global $g_t \in [0, 1]$ para o segmento correto da curva com N pontos de controlo:

- $\text{segmento} = \lfloor g_t \times N \rfloor$
- $t_{\text{local}} = g_t \times N - \text{segmento}$

Os quatro pontos de controlo são selecionados de forma cíclica:

- $\text{indices} = [(\text{seg} - 1) \bmod N, \text{seg} \bmod N, (\text{seg} + 1) \bmod N, (\text{seg} + 2) \bmod N]$

O parâmetro global é calculado a partir do tempo decorrido:

- $g_t = \frac{\text{elapsed mod time}}{\text{time}}$

No ficheiro XML, esta transformação é definida como:

```
<translate time="10" align="True">
  <point x="..." y="..." z="..." />
  <point x="..." y="..." z="..." />
  ...
</translate>
```

O atributo `time` define o número de segundos para percorrer a curva completa. São necessários no mínimo 4 pontos de controlo.

A trajetória é renderizada como uma linha (`GL_LINE_LOOP`), mostrando 100 pontos ao longo da curva, permitindo a visualização do percurso do objeto.

3.2.2. Alinhamento à Curva

Quando o atributo align está ativo ("True"), o objeto é orientado de modo a acompanhar a direção do movimento. Para tal, é construído um referencial ortonormado a partir da derivada da curva.

Sendo D a derivada normalizada no ponto atual e Y_{prev} o vetor "cima" do frame anterior:

- Eixo X (frente):

$$X = \frac{D}{\|D\|}$$

- Eixo Z (lateral): produto vetorial entre X e o Y anterior:

$$Z = \frac{X \times Y_{\text{prev}}}{\|X \times Y_{\text{prev}}\|}$$

- Eixo Y (cima): produto vetorial entre Z e X:

$$Y = \frac{Z \times X}{\|Z \times X\|}$$

O vetor Y é propagado entre frames (Y_{prev}) para garantir continuidade e evitar rotações bruscas. A matriz de rotação resultante é:

$$R = \begin{pmatrix} X_0 & X_1 & X_2 & 0 \\ Y_0 & Y_1 & Y_2 & 0 \\ Z_0 & Z_1 & Z_2 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Esta matriz é aplicada via `glMultMatrixf` após a translação, garantindo que o modelo se orienta na direção do movimento ao longo da curva.

3.2.3. Rotação Baseada em Tempo

A rotação temporal permite que um objeto complete uma rotação de 360° em torno de um eixo especificado num determinado período de tempo. O ângulo instantâneo é calculado por:

$$\theta = \left(\frac{\text{elapsed mod time}}{\text{time}} \right) \times 360^\circ$$

No ficheiro XML, esta transformação é definida por:

```
<rotate time="20" x="0" y="1" z="0" />
```

Este exemplo define uma rotação completa em torno do eixo Y a cada 20 segundos.

3.3. Renderização com VBOs

A migração de Immediate Mode para Vertex Buffer Objects (VBOs) constitui uma das alterações mais significativas desta fase. Os VBOs permitem armazenar a geometria diretamente na memória da GPU, eliminando a necessidade de transferir dados do CPU em cada frame.

3.3.1. Criação dos Buffers

Durante o parsing do XML, ao carregar cada modelo .3d, os vértices são transferidos para um buffer na GPU:

- A geometria é lida do ficheiro .3d para um vetor de **Points**
- Os pontos são convertidos num array contíguo de floats (x, y, z por vértice)
- É gerado um buffer OpenGL com `glGenBuffers`
- Os dados são enviados para a GPU com `glBufferData`, usando `GL_STATIC_DRAW`

Após esta operação, o vetor de pontos em RAM pode ser libertado, ficando apenas o identificador do buffer (vboId) e a contagem de vértices.

Para utilizar **VBOs**, é necessária a biblioteca **GLEW**, que é inicializada no arranque da aplicação com `glewInit()`, após a criação do contexto **OpenGL**.

3.3.2. Desenho dos Modelos

O desenho de cada modelo é realizado em três passos:

- `glBindBuffer(GL_ARRAY_BUFFER, vboId)` — ativa o buffer do modelo
- `glVertexPointer(3, GL_FLOAT, 0, 0)` — indica que cada vértice tem 3 componentes float
- `glDrawArrays(GL_TRIANGLES, 0, vertexCount)` — desenha todos os triângulos

A cor de cada modelo é aplicada individualmente com `glColor3f` antes do desenho, permitindo que cada corpo celeste tenha uma cor distinta.

3.4. Exploração da Cena

As funcionalidades de exploração interativa da cena foram mantidas relativamente à fase anterior:

- **Setas do teclado:** movimento orbital da câmara (ângulos α e β)
- **Tecla i/o:** zoom in/out (ajuste do camRadius)
- **Tecla a:** ativa/desativa os eixos de referência
- **Tecla m:** alterna entre wireframe, pontos e preenchido
- **Tecla c:** ativa/desativa a visualização das curvas de Catmull-Rom (trajetória do cometa)
- **Tecla r:** reinicia a câmara para a posição definida no XML

A animação contínua é garantida pelo registo de `glutIdleFunc` com a função de renderização, que provoca o redesenho constante da cena e a atualização das transformações temporais.

4. Cena de Demonstração: Sistema Solar Dinâmico

A cena de demonstração desta fase consiste numa representação dinâmica do sistema solar, onde todos os corpos celestes possuem movimento.

O Sol permanece na origem da cena com rotação própria. Os planetas orbitam o Sol através de rotações temporais combinadas com translações, e possuem também rotação própria. Vários planetas incluem subgrupos para representar luas e outros satélites.

O elemento mais significativo desta cena é o **cometa**, modelado a partir de patches de Bézier (o teapot) e animado ao longo de uma trajetória definida por uma curva de **Catmull-Rom** com 8 pontos de controlo. O alinhamento do cometa à curva está ativo, fazendo com que o objeto se oriente na direção do movimento. A trajetória é renderizada como uma linha cinzenta, visualizando o percurso orbital do cometa.

Cada corpo celeste possui uma cor distinta, definida através da tag `<color>` no XML, facilitando a identificação visual dos diferentes elementos da cena.

Entre os exemplos mais relevantes desta hierarquia destacam-se:

- A Lua como subgrupo da Terra;
- Fobos e Deimos como subgrupos de Marte;
- Quatro luas associadas a Júpiter (Io, Europa, Ganímedes, Calisto);
- Um sistema próprio para Saturno, incluindo o corpo principal, anel e Titã;
- Um cometa (teapot de Bézier) com trajetória de Catmull-Rom visível.

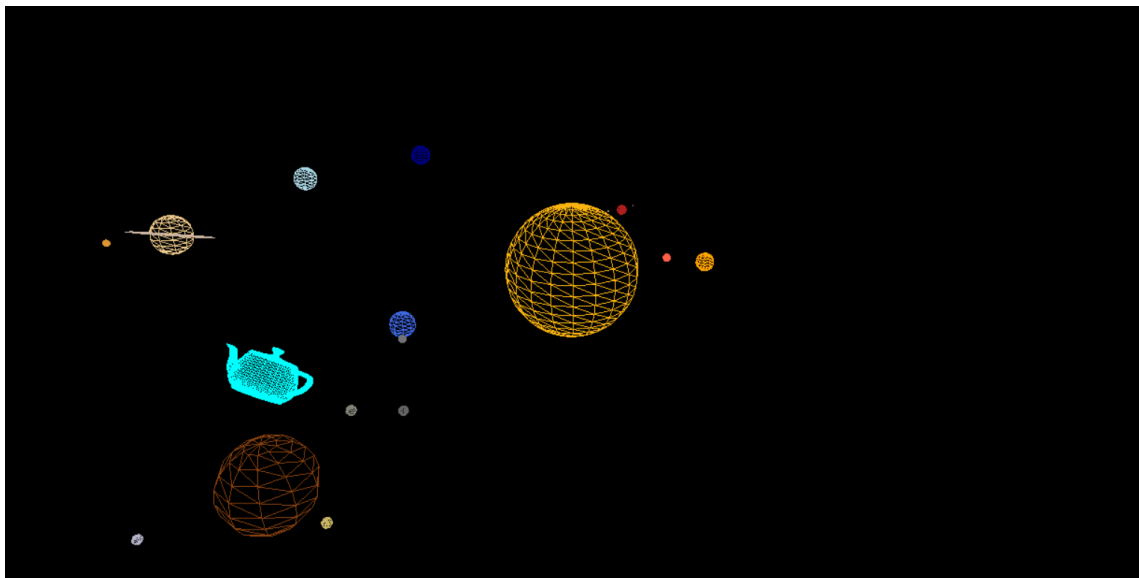


Figura 1: Vista aproximada do sistema solar com o cometa (teapot de Bézier) visível

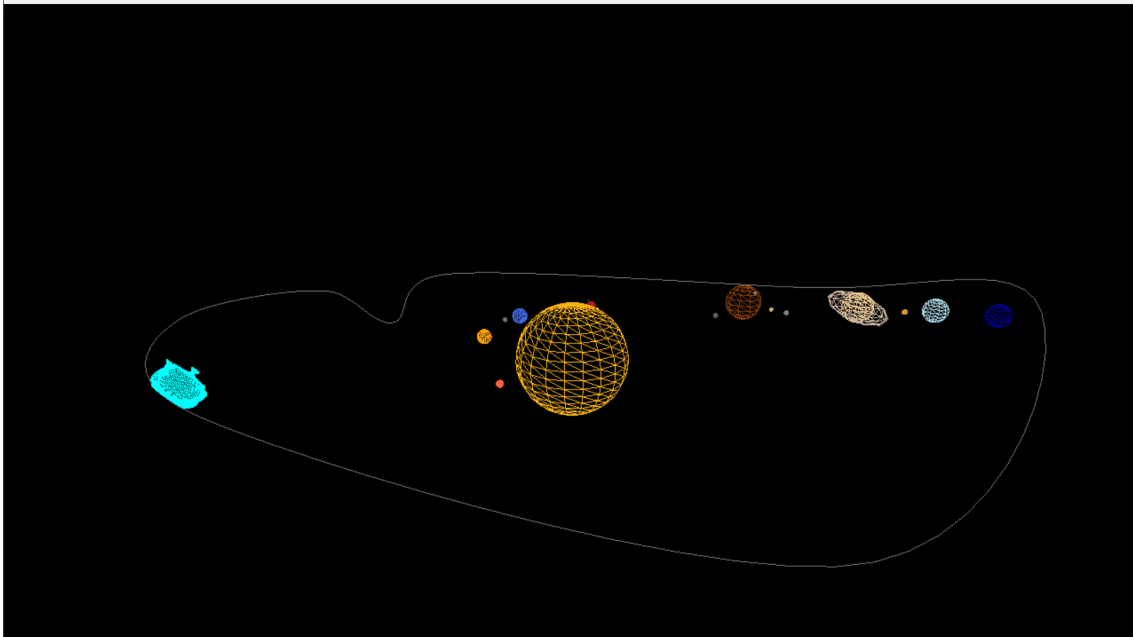


Figura 2: Vista geral do sistema solar dinâmico com a trajetória de Catmull-Rom do cometa

A tabela seguinte resume a estrutura hierárquica da cena, indicando para cada corpo celeste o modelo utilizado, as transformações aplicadas e os eventuais subgrupos associados. As transformações são apresentadas pela ordem em que são aplicadas, dado que a sua composição não é comutativa.

Corpo	Modelo	Transformações	Subgrupos
Sol	sphere (r=1, 20×20)	rotate(time=20) -> scale 5.0	—
Mercúrio	sphere (r=1, 10×10)	rotate(time=8) -> translate -> rotate(time=5) -> scale 0.3	—
Vénus	sphere (r=1, 10×10)	rotate(time=12) -> translate -> rotate(time=7) -> scale 0.7	—
Terra	sphere (r=1, 10×10)	rotate(time=15) -> translate	Lua
Marte	sphere (r=1, 10×10)	rotate(time=20) -> translate	Fobos, Deimos
Júpiter	sphere (r=1, 10×10)	rotate(time=35) -> translate	Io, Europa, Ganímedes, Calisto
Saturno	sphere (r=1, 10×10)	rotate(time=50) -> translate	Anel, Titã
Urano	sphere (r=1, 10×10)	rotate(time=60) -> translate -> scale 1.3	—
Neptuno	sphere (r=1, 10×10)	rotate(time=70) -> translate -> scale 1.25	—
Cometa	teapot (bezier, tess=10)	translate(catmull-rom, time=10, align) -> scale 0.8	—

Tabela 2: Estrutura hierárquica do sistema solar dinâmico

5. Utilitários e Estruturas de Dados

Para suportar a leitura e interpretação dos ficheiros de configuração, continuou a ser utilizada a biblioteca **RapidXML**. O processo de parsing foi estendido para reconhecer os novos atributos temporais (`time` , `align`) e para criar VBOs durante o carregamento dos modelos.

Para a utilização de VBOs, foi integrada a biblioteca **GLEW**, que permite aceder às extensões OpenGL necessárias para a gestão de buffers na GPU (`glGenBuffers` , `glBindBuffer` , `glBufferData`). A integração envolveu a adição dos ficheiros da biblioteca à pasta toolkits e a configuração do CMakeLists.txt para incluir os headers, ligar a biblioteca estática e copiar a DLL automaticamente após o build.

Ao nível das estruturas de dados partilhadas, manteve-se a utilização da classe **Point** para representar coordenadas tridimensionais. As funções **saveToFile** e **parse3Dfile** continuam a ser utilizadas para a escrita e leitura de ficheiros .3d, respetivamente.

A classe **Group** foi reformulada para armazenar uma lista de transformações individuais (em vez de uma matriz acumulada), uma lista de ModelData (com referências a VBOs) e os subgrupos filhos.

6. Conclusões e Trabalho Futuro

O desenvolvimento da terceira fase permitiu expandir significativamente as capacidades do motor gráfico, introduzindo três funcionalidades fundamentais: a geração de modelos a partir de superfícies cúbicas de Bézier, transformações animadas baseadas em tempo, e renderização otimizada através de VBOs.

A geração de superfícies de Bézier a partir de ficheiros .patch permite criar modelos complexos como o teapot, validando a implementação dos polinómios de Bernstein e da tesselação paramétrica. As transformações dinâmicas — translação por curvas de Catmull-Rom e rotação temporal — permitem animar a cena de forma contínua e fluida. A migração para VBOs elimina a necessidade de transferir geometria do CPU para a GPU em cada frame, melhorando significativamente a eficiência da renderização.

A cena de demonstração do sistema solar dinâmico valida o correto funcionamento de todas as novas funcionalidades, apresentando planetas em órbita com rotação própria e um cometa modelado por patches de Bézier a percorrer uma trajetória de Catmull-Rom.

Como trabalho futuro, prevê-se a continuação da evolução do projeto na fase seguinte, nomeadamente com a introdução de iluminação (normais nos modelos), texturas e materiais, bem como a expansão do formato dos ficheiros .3d para incluir normais e coordenadas de textura por vértice.